

Math & Code Foundations for AI

KAUST–Mawhiba IOAI Training

Week 1 · Day 1



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

April 26, 2026

1. Part I — Math Foundations

1. Linear Algebra Basics
2. Matrix & Vector Operations
3. Calculus for AI

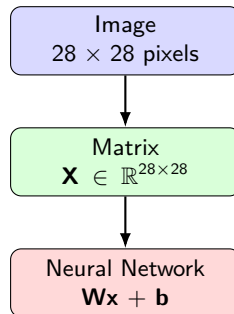
2. Part II — Code Foundations

4. Python Basics
5. Functions & OOP
6. NumPy — Numerical Python
7. Pandas & Data Science Basics

- ▶ **Data is numbers in grids:** every image, every table, every sentence (once encoded) is a matrix
- ▶ **Models are matrix operations:** neural networks multiply inputs by weight matrices
- ▶ **Training = optimization in vector space:** gradients, updates, embeddings — all linear algebra

Key Insight

If you understand vectors and matrices, you understand the language AI speaks.



- ▶ A **scalar** is a single number
- ▶ Notation: lowercase italic — a , x , α
- ▶ Scalars live in a number set:
 - $x \in \mathbb{N}$ — natural numbers $(0, 1, 2, \dots)$
 - $x \in \mathbb{Z}$ — integers $(\dots, -1, 0, 1, \dots)$
 - $x \in \mathbb{R}$ — real numbers (continuous line)

Examples in AI

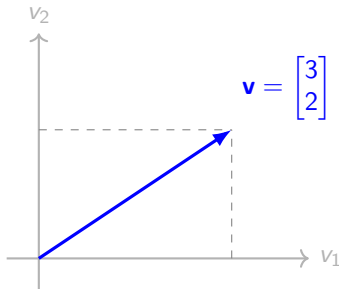
Learning rate $\alpha = 0.001$, **Loss value** $J = 2.37$, **A single pixel** $p = 142$

- ▶ A **vector** is an ordered list of numbers
- ▶ Notation: bold lowercase — **v**, **x**
- ▶ A vector with n elements lives in $\mathbb{R}^n$

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n$$

- ▶ v_i is the i -th **component** (or element)
- ▶ Vectors can be *column vectors* (default) or *row vectors* ($\mathbf{v}^\top$)

Geometric view ($\mathbb{R}^2$)



A vector is both a *point* and a *direction + magnitude*.

- ▶ **Feature vector:** a single data sample's measurements

$$\mathbf{x} = \begin{bmatrix} \text{height} \\ \text{weight} \\ \text{age} \end{bmatrix} = \begin{bmatrix} 170 \\ 65 \\ 17 \end{bmatrix} \in \mathbb{R}^3$$

- ▶ **Weight vector:** learnable parameters in a linear model

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

- ▶ **Embedding vector:** a word or image compressed into a fixed-size vector
 - CLIP maps images *and* text into the same 512-dimensional vector space

- ▶ A **matrix** is a 2D grid of numbers
- ▶ Notation: bold uppercase — **A**, **W**
- ▶ An $m \times n$ matrix has m rows and n columns

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

Reading a Matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Shape: 2×3

$a_{1,2} = 2$ (row 1, col 2)

$a_{2,3} = 6$ (row 2, col 3)

Key connection: A vector is just a matrix with one column ($m \times 1$) or one row ($1 \times n$).

- ▶ **Dataset:** N samples with d features $\Rightarrow$ matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$
 - Each row is one sample's feature vector
 - Each column is one feature across all samples

- ▶ **Grayscale image:** matrix of pixel intensities $\in \mathbb{R}^{H \times W}$

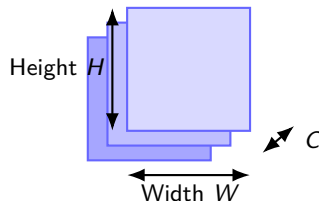
- ▶ **Weight matrix:** connects layers in a neural network

$$\mathbf{h} = f(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad \mathbf{W} \in \mathbb{R}^{\text{out} \times \text{in}}$$

- ▶ **Attention matrix:** in Transformers, $\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)$ is a matrix encoding which words attend to which

- A **tensor** generalizes scalars, vectors, and matrices to any number of dimensions (called *axes* or *ranks*)

Object	Rank	Shape example
Scalar	0	—
Vector	1	$(d,)$
Matrix	2	(m, n)
3D Tensor	3	(C, H, W)
4D Tensor	4	(B, C, H, W)



A **color image** is a 3D tensor:
 $C \times H \times W$
(e.g. $3 \times 224 \times 224$ for RGB)

A **batch** of images adds a 4th axis:
 $B \times C \times H \times W$

- ▶ The **transpose** flips rows and columns: $(\mathbf{A}^\top)_{ij} = A_{ji}$

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}_{2 \times 3} \implies \mathbf{A}^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}_{3 \times 2}$$

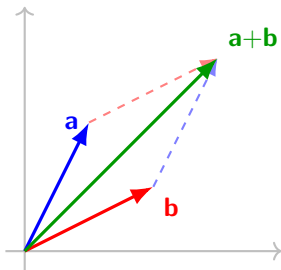
- ▶ A column vector $\mathbf{v} \in \mathbb{R}^{n \times 1}$ becomes a row vector $\mathbf{v}^\top \in \mathbb{R}^{1 \times n}$
- ▶ Useful properties:
 - $(\mathbf{A}^\top)^\top = \mathbf{A}$
 - $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$

Why it matters

The dot product $\mathbf{w} \cdot \mathbf{x}$ is written as $\mathbf{w}^\top \mathbf{x}$ in matrix notation — you will see this everywhere in ML.

Addition (element-wise, same shape):

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$$



Scalar multiplication (scale every element):

$$3 \cdot \begin{bmatrix} 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 6 \\ -3 \end{bmatrix}$$

- ▶ Scales the magnitude but keeps (or reverses) direction
- ▶ In ML: *learning rate* α scales the gradient vector before updating weights

- ▶ Element-wise: both matrices must have the **same shape**

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

- ▶ Subtraction works the same way: $(\mathbf{A} - \mathbf{B})_{ij} = a_{ij} - b_{ij}$

In neural networks

Weight update: $\mathbf{W}_{\text{new}} = \mathbf{W}_{\text{old}} - \alpha \cdot \nabla_{\mathbf{W}} J$

This is matrix subtraction — subtract the scaled gradient matrix from the weight matrix.

For two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$:

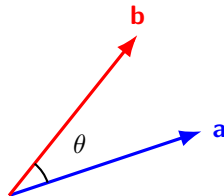
$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^\top \mathbf{b} = \sum_{i=1}^n a_i b_i$$

Example:

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}^\top \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$$

- ▶ Result is a **scalar** (single number)
- ▶ Measures *similarity*: larger $\Rightarrow$ more aligned

Geometric interpretation



$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

$\theta = 0^\circ$: max similarity

$\theta = 90^\circ$: orthogonal (unrelated)

$\theta = 180^\circ$: opposite

The dot product is **the most fundamental operation in deep learning**:

1. **Linear prediction:** $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$ (every neuron computes a dot product)
2. **Cosine similarity:** $\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$ (used in CLIP, search, recommendation)
3. **Attention scores:** $\text{score} = \mathbf{q}^\top \mathbf{k}$ (every query–key pair in a Transformer)

Every time you hear “similarity” or “score” in AI, there is almost certainly a dot product underneath.

For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$, the product $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}$:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} = (\text{row } i \text{ of } \mathbf{A}) \cdot (\text{col } j \text{ of } \mathbf{B})$$

Example:

$$\underbrace{\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}}_{2 \times 2} \underbrace{\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}}_{2 \times 2} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

$$c_{11} = 1 \cdot 5 + 2 \cdot 7 = 19$$

Shape rule:

$$(m \times \underbrace{n}_{\text{must match}}) \times (\underbrace{n}_{\text{must match}} \times p) = (m \times p)$$

- ▶ Inner dimensions must match
- ▶ Outer dimensions give the result shape

- **Neural network layer:** maps input vector to output vector

$$\mathbf{h} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad \underbrace{\mathbf{W}}_{\text{out} \times \text{in}} \underbrace{\mathbf{x}}_{\text{in} \times 1} = \underbrace{\mathbf{h}}_{\text{out} \times 1}$$

- **Batch processing:** multiply weight matrix by an entire batch at once

$$\underbrace{\mathbf{W}}_{d_{\text{out}} \times d_{\text{in}}} \underbrace{\mathbf{X}^{\top}}_{d_{\text{in}} \times N} = \underbrace{\mathbf{H}}_{d_{\text{out}} \times N}$$

- **Attention in Transformers:**

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}\right) \mathbf{V}$$

Three matrix multiplications in one equation!

Element-wise (Hadamard) product

⊙:

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \odot \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 4 \\ 10 \\ 18 \end{bmatrix}$$

- ▶ Same shape required
- ▶ Used in gating mechanisms (LSTM, GRU)

Vector norms (length/magnitude):

- ▶ **L2 norm** (Euclidean):

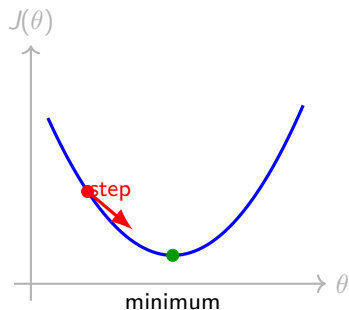
$$\|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2}$$

- ▶ **L1 norm** (Manhattan):

$$\|\mathbf{v}\|_1 = \sum_i |v_i|$$

- ▶ Used in *regularization*, *normalization*, and *distance metrics*

- ▶ AI models **learn** by minimizing a loss function
- ▶ To minimize: find where the function **decreases fastest**
- ▶ That direction is given by the *derivative* (1D) or *gradient* (n D)
- ▶ The entire training loop is *calculus in action*



The derivative tells us **which way** is downhill and **how steep**.

- ▶ The **derivative** $f'(x)$ measures the *rate of change* of f at point x

$$f'(x) = \frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- ▶ Intuition: the **slope of the tangent line**

Common derivatives:

$f(x)$	$f'(x)$
c (constant)	0
x^n	nx^{n-1}
e^x	e^x
$\ln x$	$1/x$
$\sin x$	$\cos x$

Derivative rules:

- ▶ **Sum:** $(f+g)' = f' + g'$
- ▶ **Product:** $(fg)' = f'g + fg'$
- ▶ **Constant:** $(cf)' = c \cdot f'$

Worked Example: Derivative of a Loss

Suppose a simple model predicts $\hat{y} = wx$ and we use **squared error**:

$$J(w) = (y - wx)^2$$

Expand: $J(w) = y^2 - 2ywx + w^2x^2$

Differentiate with respect to w :

$$\frac{dJ}{dw} = -2yx + 2wx^2 = 2x(wx - y) = -2x(y - \hat{y})$$

Interpretation

The gradient is proportional to the **error** $(y - \hat{y})$ and the **input** x .
Larger error $\Rightarrow$ bigger update. Zero error $\Rightarrow$ no update. This is exactly how learning works!

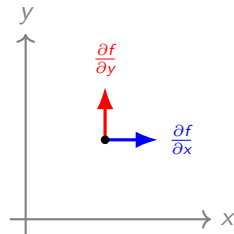
When a function has **multiple variables**, the *partial derivative* measures the rate of change **with respect to one variable**, holding all others fixed.

Example: $f(x, y) = x^2 + 3xy + y^2$

$$\frac{\partial f}{\partial x} = 2x + 3y$$

$$\frac{\partial f}{\partial y} = 3x + 2y$$

- ▶ $\frac{\partial f}{\partial x}$: how does f change when we nudge x alone?
- ▶ $\frac{\partial f}{\partial y}$: how does f change when we nudge y alone?
- ▶ Neural nets have *millions* of parameters — we need a partial derivative for each one



Each partial derivative looks at change along **one axis only**.

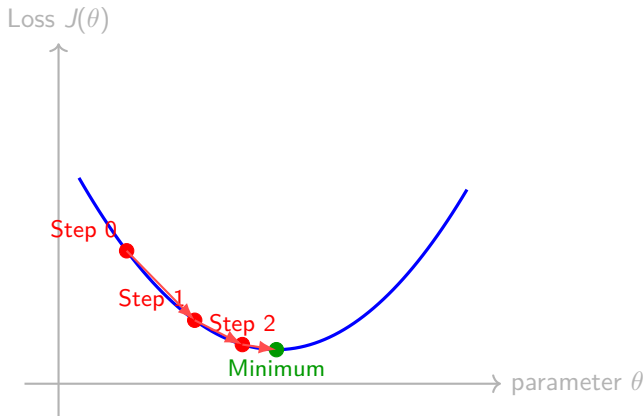
The **gradient** collects *all* partial derivatives into a single vector:

$$\nabla_{\theta} J = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \\ \vdots \\ \frac{\partial J}{\partial \theta_n} \end{bmatrix} \in \mathbb{R}^n$$

- ▶ The gradient points in the direction of *steepest increase* of J
- ▶ To **decrease** J , move in the *opposite direction*: $-\nabla J$
- ▶ This is exactly **Gradient Descent**:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla_{\theta} J$$

α is the **learning rate**. Too large $\Rightarrow$ overshoot. Too small $\Rightarrow$ too slow.
Choosing it well is an art.



1. Compute gradient (slope) at current position
2. Take a step in the **opposite** direction (downhill)
3. Repeat until loss stops decreasing

Neural networks are **compositions of functions**: $f(g(h(x)))$. The *chain rule* tells us how to differentiate compositions.

Single variable:

$$\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$$

Example: Let $y = (3x + 2)^4$. Set $u = 3x + 2$, so $y = u^4$.

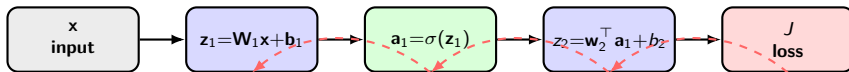
$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx} = 4u^3 \cdot 3 = 12(3x + 2)^3$$

Why it matters

Backpropagation — the algorithm that trains every neural network — is just the chain rule applied layer by layer, from the loss back to the first weight.

Chain Rule in a Neural Network

Consider a two-layer network: $\hat{y} = \sigma(\mathbf{w}_2^\top \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + b_2)$



Backward pass (backpropagation)

$$\frac{\partial J}{\partial \mathbf{W}_1} = \frac{\partial J}{\partial z_2} \cdot \frac{\partial z_2}{\partial \mathbf{a}_1} \cdot \frac{\partial \mathbf{a}_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial \mathbf{W}_1}$$

Each $\cdot$ is one link in the chain. Backpropagation computes this **right to left**, reusing intermediate results. PyTorch does it for you automatically!

Concept	Notation	Where in AI
Scalar	x, α	Learning rate, loss value
Vector	$\mathbf{v} \in \mathbb{R}^n$	Features, weights, embeddings
Matrix	$\mathbf{A} \in \mathbb{R}^{m \times n}$	Datasets, weight layers, attention
Tensor	$\mathcal{T} \in \mathbb{R}^{d_1 \times \dots \times d_k}$	Images, batches
Dot product	$\mathbf{a}^\top \mathbf{b}$	Neurons, similarity, attention
Matrix multiply	$\mathbf{AB}$	Layer forward pass, attention
Derivative	$\frac{df}{dx}$	Rate of change, slope
Partial derivative	$\frac{\partial f}{\partial x_i}$	Per-parameter sensitivity
Gradient	∇J	Direction to update all params
Chain rule	$\frac{df}{dx} = \frac{df}{du} \frac{du}{dx}$	Backpropagation

- ▶ **The language of AI:** PyTorch, TensorFlow, scikit-learn, HuggingFace — all Python
- ▶ **Readable:** designed to look like pseudocode
- ▶ **Rich ecosystem:** NumPy, Pandas, Matplotlib — one-liners that would take 50 lines in C++
- ▶ **Interactive:** Jupyter notebooks let you experiment cell by cell

Your first Python

```
print("Hello IOAI!")
```

Output: Hello IOAI!

- ▶ No semicolons
- ▶ No curly braces
- ▶ **Indentation** defines blocks

Python is **dynamically typed** — the type is inferred automatically.

```
x = 42                # int      (integer)
pi = 3.14             # float    (decimal)
name = "KAUST"        # str      (string)
is_ready = True       # bool     (True / False)
```

Type checking:

```
type(x)      # <class 'int'>
type(pi)     # <class 'float'>
```

Type conversion:

```
int(3.7)      # 3      (truncates)
float(5)      # 5.0
str(42)       # "42"
```

A **list** is an ordered, mutable collection — the workhorse of Python.

```
scores = [90, 85, 78, 92, 88]
```

```
# Access by index (0-based)
```

```
scores[0]      # 90   (first element)
```

```
scores[-1]     # 88   (last element)
```

```
# Slicing
```

```
scores[1:3]    # [85, 78] (index 1 up to, not including, 3)
```

```
# Useful methods
```

```
scores.append(95)      # add to end
```

```
scores.sort()          # sort in place
```

```
len(scores)            # 6   (number of elements)
```

Lists can hold mixed types: ["KAUST", 42, 3.14, True] — but in data science we usually keep them uniform.

Tuple — like a list, but *immutable*
(cannot change after creation):

```
shape = (28, 28)
rgb = (255, 0, 0) # red
```

```
shape[0]      # 28
# shape[0] = 32 <- ERROR!
```

- Used for things that **shouldn't change**: dimensions, coordinates, function returns

Dictionary — key-value pairs:

```
student = {
    "name": "Sara",
    "age": 17,
    "score": 95
}
```

```
student["name"]      # "Sara"
student["age"]        # 17
```

```
# Add a new key
student["grade"] = "A+"
```

for loop — iterate over a sequence:

```
# Loop over a list
names = ["Ali", "Sara", "Omar"
        ]
for name in names:
    print(f"Hello, {name}!")

# Loop with range
for i in range(5):    #
    0,1,2,3,4
    print(i ** 2)
```

while loop — repeat while condition is true:

```
loss = 10.0
while loss > 0.1:
    loss = loss * 0.9
    print(f"Loss: {loss:.2f}")
```

List comprehension — compact loop:

```
squares = [x**2 for x in range
            (6)]
# [0, 1, 4, 9, 16, 25]
```

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(f"Grade: {grade}") # Grade: B
```

Comparison operators:

==, !=, <, >, <=, >=

Logical operators:

and, or, not

A **function** is a reusable block of code that takes inputs and returns an output.

```
def mse_loss(y_true, y_pred):  
    """Mean Squared Error --- the most basic loss function."""  
    n = len(y_true)  
    total = 0  
    for i in range(n):  
        total += (y_true[i] - y_pred[i]) ** 2  
    return total / n  
  
# Usage  
loss = mse_loss([3, 5, 7], [2.8, 5.1, 7.3])  
print(f"Loss: {loss:.4f}")    # Loss: 0.0467
```

- ▶ **def** keyword defines a function
- ▶ **Parameters:** inputs inside parentheses
- ▶ **return:** the output value

Default arguments:

```
def greet(name, greeting="Hello"):  
    return f"{greeting}, {name}!"  
  
greet("Sara")  
# "Hello, Sara!"  
greet("Ali", greeting="Marhaba")  
# "Marhaba, Ali!"
```

Lambda (anonymous) functions:

```
square = lambda x: x ** 2  
square(5) # 25  
  
# Common in sorting/mapping  
names = ["Sara", "Ali", "Omar"]  
sorted(names, key=lambda n:  
        len(n))  
# ['Ali', 'Sara', 'Omar']
```

A **class** is a blueprint. An **object** is an instance of that blueprint.

```
class Neuron:
    def __init__(self, n_inputs):
        self.weights = [0.0] * n_inputs
        self.bias = 0.0

    def forward(self, inputs):
        total = self.bias
        for w, x in zip(self.weights, inputs):
            total += w * x
        return total

n = Neuron(3)                # create a neuron with 3 inputs
n.weights = [0.5, -0.3, 0.8]
output = n.forward([1, 2, 3]) # 0.5 - 0.6 + 2.4 = 2.3
```

1. Encapsulation

Group data + behavior together:

```
class Dataset:
    def __init__(self, X, y):
        self.X = X # features
        self.y = y # labels

    def __len__(self):
        return len(self.X)
```

`__init__`: constructor (called when you create an object)

`self`: reference to *this* object

2. Inheritance

Build on existing classes:

```
class ImageDataset(Dataset):
    def __init__(self, X, y,
                    size):
        super().__init__(X, y)
        self.size = size

    def get_shape(self):
        return (len(self),
                self.size)
```

`ImageDataset` *inherits* everything from `Dataset` and adds more.

Every major AI framework is built on classes:

```
import torch.nn as nn

class MyModel(nn.Module):          # inherit from PyTorch
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 128)
        self.layer2 = nn.Linear(128, 10)

    def forward(self, x):           # define the forward pass
        x = torch.relu(self.layer1(x))
        return self.layer2(x)
```

You don't need to master OOP right now — but **recognize the pattern**: a class with `__init__` (setup) and a main method (`forward`). You'll use this pattern for every model you build.

- ▶ Python lists are **slow** for math — they weren't designed for it
- ▶ **NumPy** provides fast, memory-efficient arrays with C-speed operations
- ▶ The foundation under *every* AI library: Pandas, scikit-learn, PyTorch all use NumPy arrays internally

```
import numpy as np
```

Speed comparison

Adding 1 million numbers:

Python list: ~ 150 ms

NumPy array: ~ 1 ms

$\approx 150\times$ faster

```
# From a list
v = np.array([1, 2, 3])           # vector (1D)
A = np.array([[1, 2], [3, 4]])    # matrix (2D)

# Common constructors
np.zeros((3, 4))                  # 3x4 matrix of zeros
np.ones((2, 3))                   # 2x3 matrix of ones
np.eye(3)                         # 3x3 identity matrix
np.random.randn(5, 3)             # 5x3 random (normal dist.)
np.arange(0, 10, 2)               # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)              # [0, 0.25, 0.5, 0.75, 1.0]

# Shape & dimensions
A.shape        # (2, 2)
A.ndim         # 2
A.dtype        # float64
```

```
A = np.array([[10, 20, 30],  
              [40, 50, 60],  
              [70, 80, 90]])
```

```
A[0, 1]      # 20          (row 0, col 1)  
A[2, :]      # [70, 80, 90] (entire row 2)  
A[:, 0]      # [10, 40, 70] (entire column 0)  
A[0:2, 1:]   # [[20, 30],   (rows 0-1, cols 1+)  
              # [50, 60]]
```

Boolean indexing:

```
x = np.array([3, 7, 2, 9, 1])  
x[x > 5]      # [7, 9]
```

Reshape:

```
v = np.arange(6)      #  
      [0,1,2,3,4,5]  
v.reshape(2, 3)  
# [[0, 1, 2],  
#   [3, 4, 5]]
```



```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Element-wise operations
a + b          # [5, 7, 9]
a * b          # [4, 10, 18]    (Hadamard product)
a ** 2         # [1, 4, 9]

# Dot product
np.dot(a, b)   # 32    (1*4 + 2*5 + 3*6)
a @ b          # 32    (@ operator, same thing)

# Norm
np.linalg.norm(a)    # sqrt(1+4+9) = 3.742
```

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication
A @ B          # [[19, 22], [43, 50]]
np.matmul(A, B) # same thing

# Transpose
A.T            # [[1, 3], [2, 4]]

# Element-wise multiply (NOT matrix multiply!)
A * B          # [[5, 12], [21, 32]]
```

Common mistake: $A * B$ is element-wise, $A @ B$ is matrix multiplication.
Know the difference!

NumPy automatically **stretches** smaller arrays to match larger ones:

```
A = np.array([[1, 2, 3],  
              [4, 5, 6]])    # shape (2, 3)  
b = np.array([10, 20, 30])  # shape (3,)  
  
A + b    # [[11, 22, 33],    # b is "broadcast" to each row  
          # [14, 25, 36]]
```

Rules:

1. Dimensions compared right-to-left
2. Sizes must match or one must be 1
3. The size-1 dimension is stretched

In AI — the bias add:

```
# Batch of 32 samples, 128  
# features  
X = np.random.randn(32, 128)  
b = np.zeros(128)    # bias  
# vector  
  
# Broadcasting adds b to every  
# row  
Z = X @ W.T + b    # shape (32, 128)
```

Math	NumPy	Notes
$\mathbf{a} + \mathbf{b}$	<code>a + b</code>	Element-wise
$c \cdot \mathbf{a}$	<code>c * a</code>	Scalar $\times$ vector
$\mathbf{a}^\top \mathbf{b}$	<code>a @ b</code>	Dot product
$\mathbf{AB}$	<code>A @ B</code>	Matrix multiply
$\mathbf{A}^\top$	<code>A.T</code>	Transpose
$\ \mathbf{a}\ _2$	<code>np.linalg.norm(a)</code>	L2 norm
$\mathbf{A}^{-1}$	<code>np.linalg.inv(A)</code>	Matrix inverse
$\mathbf{a} \odot \mathbf{b}$	<code>a * b</code>	Hadamard product

Every equation from the math section can be written as **one line of NumPy**.
This is why Python dominates AI.

What is Pandas?

- ▶ Python library for **structured (tabular) data**
- ▶ Think of it as *Excel on steroids*
- ▶ Two core objects:
 - **Series**: a single column (1D)
 - **DataFrame**: a table (2D)
- ▶ Built on NumPy — fast array math under the hood

```
import pandas as pd
```

The Data Pipeline

CSV file $\xrightarrow{\text{read}}$ DataFrame $\xrightarrow{\text{clean, explore}}$ NumPy arrays $\xrightarrow{\text{train}}$ ML Model

```
# From a dictionary
data = {
    "Name":    ["Ali", "Sara", "Omar", "Lina"],
    "Age":     [17, 16, 17, 18],
    "Score":   [88, 95, 72, 91],
    "Passed":  [True, True, False, True]
}
df = pd.DataFrame(data)
```

	Name	Age	Score	Passed
0	Ali	17	88	True
1	Sara	16	95	True
2	Omar	17	72	False
3	Lina	18	91	True

Each column is a **feature**. Each row is a **sample**. This is exactly how ML datasets look.

```
# Read a CSV file into a DataFrame
df = pd.read_csv("titanic.csv")

# First look at the data
df.head()           # first 5 rows
df.shape            # (891, 12) -> 891 samples, 12 features
df.columns          # list of column names
df.dtypes           # data type of each column
df.info()           # summary: types, non-null counts
```

CSV = Comma-Separated Values. The most common format for ML datasets. Kaggle, UCI, and most real-world data comes as CSVs.

```
# Select a single column (returns a Series)
df["Age"]

# Select multiple columns (returns a DataFrame)
df[["Name", "Score"]]

# Filter rows with a condition
df[df["Score"] > 80]          # only high scorers
df[df["Age"] == 17]          # only age 17

# Combine conditions
df[(df["Score"] > 80) & (df["Age"] >= 17)]
```

Access by position:

```
df.iloc[0]          # first row
df.iloc[0:3, 1:3]    # rows 0-2,
                     cols 1-2
```

Access by label:

```
df.loc[0, "Name"]    # "Ali"
```



```
# Summary statistics for all numeric columns  
df.describe()
```

```
# Individual statistics
```

```
df["Score"].mean()      # 86.5  
df["Score"].std()       # standard deviation  
df["Score"].min()       # 72  
df["Score"].max()       # 95  
df["Score"].median()    # 89.5
```

```
# Value counts (for categorical columns)
```

```
df["Passed"].value_counts()  
# True      3  
# False     1
```

- ▶ `.describe()` gives count, mean, std, min, 25%, 50%, 75%, max — your **first step** on any new dataset

Real-world datasets almost always have **missing values**:

```
# Detect missing values
df.isnull().sum()          # count NaNs per column

# Strategy 1: Drop rows with missing values
df_clean = df.dropna()

# Strategy 2: Fill with a value
df["Age"].fillna(df["Age"].mean(), inplace=True)

# Strategy 3: Fill with forward/backward fill
df["Score"].fillna(method="ffill", inplace=True)
```

Rule of thumb

Few missing values ($< 5\%$)? Fill them.

Many missing ($> 40\%$)? Consider dropping the column.

Always **check and report** missing data before modeling.

- ▶ ML models expect **NumPy arrays**, not DataFrames
- ▶ Separate **features** (X) from **target** (y)

```
# Features: everything except the target column
X = df[["Age", "Score"]].values          # NumPy array

# Target: the column we want to predict
y = df["Passed"].astype(int).values      # NumPy array

print(X.shape)    # (4, 2)    -> 4 samples, 2 features
print(y.shape)    # (4,)      -> 4 labels
```

The complete pipeline: CSV $\rightarrow$ `pd.read_csv` $\rightarrow$ explore & clean $\rightarrow$ `.values`
 $\rightarrow$ NumPy $\rightarrow$ model. You will do this for **every** project.

Credits

KAUST Academy

King Abdullah University of Science and Technology

KAUST–Mawhiba IOAI 2026 Training Program